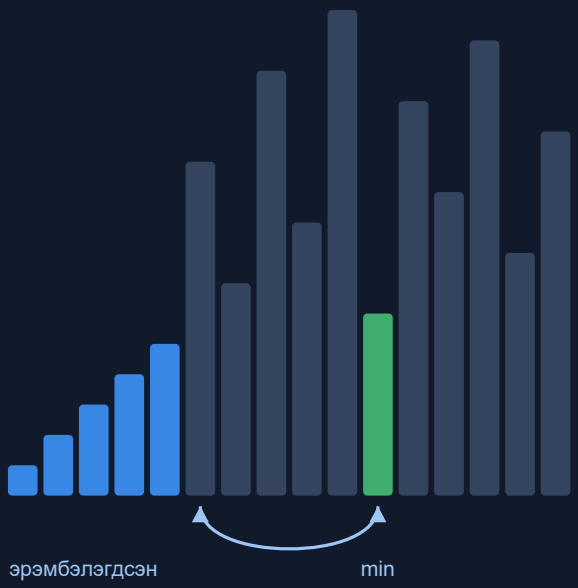
АЛГОРИТМЫН ҮНДЭС · PYTHON · ЭРЭМБЭЛЭЛТ 1/3

Selection sort

Сонгон эрэмбэлэлт: хамгийн багыг сонгож, урд нь тавих



Түвшин: Ойлголт + хэрэгжүүлэлт | **Жишээ:** Python 3 | **Урьдчилсан мэдлэг:** list, for давталт, Linear search, Big O

Энэ хичээлийг уншсаны дараа та:

- Эрэмбэлэлт гэж юу болох, **яагаад** хэрэгтэйг ойлгоно.
- Selection sort-ийн санааг **зураг, хүснэгтээр** тайлбарлана.
- Selection sort-ийг **өөрөө бичиж**, гараар мөрдөж шалгана.
- Яагаад **O(n²)** болохыг харьцуулалтын тоогоор тооцож, бодит хэмжилтээр харна.
- 3 түгээмэл алдааг таньж, кодоо `sorted()`-тэй харьцуулж **тестлэнэ**.

Агуулга

01 Эрэмбэлэлт гэж юу вэ?
02 Санаа
03 Код
04 Гараар мөрдөх
05 Түгээмэл алдаа
06 Хэр хурдан бэ?
07 Тестлэх
08 Дасгал
09 Хураангуй

> **Нэг өгүүлбэрээр**
> **Selection sort** (сонгон эрэмбэлэлт): эрэмбэлэгдээгүй хэсгээс хамгийн багыг **сонгож**, тэр хэсгийн эхэнд тавь; үүнийг n − 1 удаа давт.

01 Эрэмбэлэлт гэж юу вэ?

Эрэмбэлэх гэдэг нь элементүүдийг тодорхой дараалаар — ихэвчлэн **өсөхөөр** (багаас их рүү) — байрлуулах явдал. Тоо, үг, оноо, огноо… харьцуулж болох бүх зүйлийг эрэмбэлж болно:

Өмнө	Дараа	Юугаар
`[5, 2, 9, 1, 7]`	`[1, 2, 5, 7, 9]`	тоо, өсөхөөр
`[5, 2, 9, 1, 7]`	`[9, 7, 5, 2, 1]`	тоо, буурахаар
`["Сараа", "Бат", "Дорж"]`	`["Бат", "Дорж", "Сараа"]`	үг, цагаан толгойгоор

Яагаад эрэмбэлэх вэ?

- **Хурдан хайх.** Binary search зөвхөн эрэмбэлэгдсэн жагсаалт дээр ажилладаг (Хайлт 2/2). Нэг удаа эрэмбэлээд дараа нь олон удаа O(log n)-ээр хайна.
- **Хүмүүст ойлгомжтой.** Онооны жагсаалт, үнээр нь эрэмбэлсэн бараа, утасны дэвтэр.
- **Бусад бодлогыг хялбарчлах.** Хамгийн бага/их k элемент, медиан, давхардал олох — эрэмбэлсний дараа хөрш элементүүдийг харахад л хангалттай.

Python-д бэлэн байгаа

```
nums = [5, 2, 9, 1, 7]

new = sorted(nums)      # ШИНЭ жагсаалт буцаана, nums өөрчлөгдөхгүй
print(new)              # [1, 2, 5, 7, 9]

nums.sort()             # БАЙРАНДАА эрэмбэлнэ, None буцаана
print(nums)             # [1, 2, 5, 7, 9]
```

> **Тэгвэл яагаад өөрсдөө бичих вэ?**
> Ажлын кодонд бэлэн функцийг хэрэглэнэ. Харин эрэмбэлэх алгоритм бол алгоритмын сэтгэлгээний хамгийн сайн дасгал: давталт, индекс, солих үйлдэл, Big O — бүгд нэг дор. Энэ цуврал 3 алгоритм үзнэ: **Selection sort** (энэ хичээл), **Insertion sort**, **Merge sort**. Эхний хоёр нь энгийн боловч O(n²), гурав дахь нь O(n log n).

Бидний бичих `selection_sort(items)` функц `list.sort()`-ийн адил жагсаалтыг **байрандаа** өсөхөөр эрэмбэлж, юу ч буцаахгүй.

02 Санаа

Зүйрлэл: биеийн тамирын багш сурагчдыг намаас өндөр нь жагсаана. Бүгдээс хамгийн намыг нь олж 1-р байранд, үлдсэнээс хамгийн намыг нь 2-р байранд… гэж **сонгосоор** байна.

Жагсаалтыг үргэлж хоёр хэсэгтэй гэж бод:

- **Зүүн хэсэг** — эрэмбэлэгдсэн, **эцсийн** байрандаа орсон элементүүд. Эхэндээ хоосон.
- **Баруун хэсэг** — эрэмбэлэгдээгүй. Эхэндээ бүх жагсаалт.

Эргэлт бүрт баруун хэсгээс хамгийн багыг олж (linear search шиг нэг бүрчлэн харна), баруун хэсгийн эхний элементтэй **байрыг нь солино**. Ингэснээр зүүн хэсэг нэгээр томорно.



Зураг 1. [29, 10, 14, 37, 13]-ийг selection sort-оор эрэмбэлэх нь. Ногоон — эрэмбэлэгдсэн хэсэг, хар хөх — олдсон хамгийн бага, цайвар хөх — эрэмбэлэгдээгүй хэсэг, улбар шар нум — байраа солих хоёр элемент. 5 элементэд 4 эргэлт.

> **Яагаад n − 1 эргэлт хангалттай вэ?**
> n − 1 элемент эцсийн байрандаа орсны дараа үлдсэн ганц элемент нь хамгийн их нь байх нь гарцаагүй — түүний байр аль хэдийн зөв.

03 Код

Алхам 1. Эрэмбэлэгдээгүй хэсгийн эхлэл `i` 0-ээс n − 2 хүртэл гүйнэ: `for i in range(n - 1):`

Алхам 2. Хамгийн багыг одоохондоо эхний элемент гэж үзнэ: `min_i = i` — **индексийг** нь хадгална (утгыг биш), учир нь дараа нь солих хэрэгтэй.

Алхам 3. Үлдсэн элементүүдийг нэг бүрчлэн харж, илүү багыг олбол индексийг шинэчилнэ: `for j in range(i + 1, n):`

Алхам 4. Дотоод давталт дууссаны **дараа** ганц удаа солино: `items[i], items[min_i] = items[min_i], items[i]`

```
def selection_sort(items):
    n = len(items)
    for i in range(n - 1):              # 1. эрэмбэлэгдээгүй хэсгийн эхлэл
        min_i = i                       # 2. хамгийн багын ИНДЕКС
        for j in range(i + 1, n):       # 3. үлдсэнийг нэг бүрчлэн харна
            if items[j] < items[min_i]:
                min_i = j
        items[i], items[min_i] = items[min_i], items[i]   # 4. солино

nums = [29, 10, 14, 37, 13]
selection_sort(nums)        # байрандаа эрэмбэлнэ — юу ч буцаахгүй
print(nums)                 # [10, 13, 14, 29, 37]
```

Python-д хоёр утгыг солих

Ихэнх хэлэнд солихын тулд түр хувьсагч хэрэгтэй. Python-д нэг мөрөөр болно: баруун тал эхлээд бүхэлдээ бодогдож, дараа нь зүүн талд оноогдоно.

Түр хувьсагчтай (бусад хэлэнд)

```
tmp = a
a = b
b = tmp
```

Python-д

```
a, b = b, a
# жагсаалтын хоёр элемент:
items[i], items[k] = items[k], items[i]
```

> **Linear search-тэй холбоо**
> 3-р алхам бол «хамгийн багыг олох» linear search: үлдсэн хэсгийг эхнээс нь дуустал харна. Тэгэхээр selection sort = «хамгийн багыг хайх»-ыг n − 1 удаа давтах.

04 Гараар мөрдөх

Эхлээд нэг эргэлтийн дотоод давталтыг нарийвчлан мөрдье: `i = 1`, жагсаалт `[10, 29, 14, 37, 13]`. `min_i` нь `1`-ээс (утга 29) эхэлнэ.

j	items[j]	items[min_i]	items[j] < items[min_i] ?	min_i
2	14	29	**True**	**2**
3	37	14	False	2
4	13	14	**True**	**4**
—			`items[1] ⇔ items[4]`	`[10, 13, 14, 37, 29]`

Хамгийн бага 13 нь индекс 4-т байжээ. Эргэлтийн төгсгөлд нэг удаа солиход зүүн хэсэг [10, 13] болж томорлоо.

Бүтэн эрэмбэлэлт

i	Эргэлтийн өмнө	min (индекс)	Солих	Эргэлтийн дараа	Харьц.
0	`[29, 10, 14, 37, 13]`	10 (1)	0 ↔ 1	`[10, 29, 14, 37, 13]`	4
1	`[10, 29, 14, 37, 13]`	13 (4)	1 ↔ 4	`[10, 13, 14, 37, 29]`	3
2	`[10, 13, 14, 37, 29]`	14 (2)	солихгүй	`[10, 13, 14, 37, 29]`	2
3	`[10, 13, 14, 37, 29]`	29 (4)	3 ↔ 4	`[10, 13, 14, 29, 37]`	1
				Нийт	**10**

Нийт харьцуулалт: 4 + 3 + 2 + 1 = **10**. Жинхэнэ солилт: **3** — i = 2 үед хамгийн бага нь аль хэдийн байрандаа байсан тул солих мөр элементийг өөртэй нь солиод юу ч өөрчлөөгүй.

> **Зөвлөгөө: компьютерээр мөрдүүлэх**
> Гадна давталтын төгсгөлд нэг мөр нэмбэл хүснэгтийн баруун баганыг Python өөрөө хэвлэнэ:

```
items[i], items[min_i] = items[min_i], items[i]
print(f"i={i}  min={items[i]}  {items}")
```

05 Түгээмэл алдаа

Эхлэгчдийн хамгийн их гаргадаг 3 алдаа. Бүгдийг бодитоор ажиллуулж шалгав.

Алдаа 1 · min_i-г 0-ээс эхлүүлэх

БУРУУ

```
for i in range(n - 1):
    min_i = 0
    ...
```

ЗӨВ

```
for i in range(n - 1):
    min_i = i
    ...
```

Юу болох вэ: `selection_sort([3, 1, 2])`-ийн дараа жагсаалт `[3, 1, 2]` — **огт өөрчлөгдөөгүй**!

Яагаад: i = 1 үед хайлт 0-р байрнаас эхэлнэ. Тэнд аль хэдийн байрандаа орсон хамгийн бага элемент байгаа тул түүнээс бага юм олдохгүй — min_i = 0 хэвээр үлдэж, 0-р элемент i-тэй солигдоод эрэмбэлэгдсэн хэсгийг эвдэнэ. Хамгийн багыг зөвхөн **эрэмбэлэгдээгүй** хэсгээс хайна.

Алдаа 2 · items[min_i]-ийн оронд items[i]-тэй харьцуулах

БУРУУ

```
for j in range(i + 1, n):
    if items[j] < items[i]:
        min_i = j
```

ЗӨВ

```
for j in range(i + 1, n):
    if items[j] < items[min_i]:
        min_i = j
```

Юу болох вэ: `selection_sort([5, 3, 1, 4])` → `[4, 1, 3, 5]`.

Яагаад: Энэ нь items[i]-ээс бага **сүүлийн** элементийг олдог, хамгийн багыг биш: 3, 1, 4 гурвуулаа 5-аас бага тул min_i эцэст нь 4-ийн индекс болно. Одоогийн хамгийн багатай (items[min_i]) харьцуулах ёстой.

Алдаа 3 · Үр дүнг хувьсагчид оноох

БУРУУ

```
nums = selection_sort(nums)
print(nums)
```

ЗӨВ

```
selection_sort(nums)
print(nums)
```

Юу болох вэ: `None` хэвлэгдэнэ — эрэмбэлсэн жагсаалт алга болно.

Яагаад: Функц байрандаа эрэмбэлээд юу ч буцаадаггүй (`return`-гүй функц `None` буцаана). `nums.sort()` мөн адил: `nums = nums.sort()` бол түгээмэл алдаа. Шинэ жагсаалт хэрэгтэй бол `sorted(nums)`.

06 Хэр хурдан бэ?

Ажлыг **харьцуулалтын тоогоор** тоолъё. `i`-р эргэлтэд дотоод давталт n − 1 − i удаа харьцуулна:

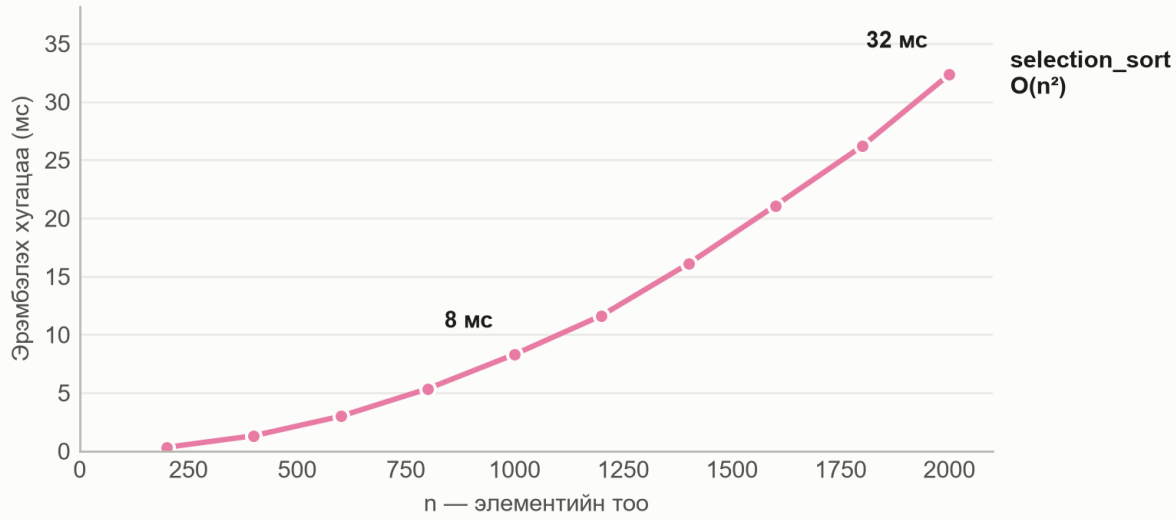
$$(n - 1) + (n - 2) + \ldots + 2 + 1 = \mathbf{n(n - 1) / 2 \approx n^2 / 2}$$

n	Харьцуулалт n(n − 1) / 2	Солилт (хамгийн ихдээ n − 1)
10	45	9
100	4 950	99
1 000	499 500	999
1 000 000	499 999 500 000 (≈ 5 · 10^{11})	999 999

Коэффициентийг хаяхад **O(n²)** (Big O хичээлийн «давталт дотор давталт» дүрэм). Анхаарах зүйл: жагсаалт аль хэдийн эрэмбэлэгдсэн байсан ч selection sort яг **ижил** тооны харьцуулалт хийнэ — хамгийн багыг олохын тулд үлдсэнийг заавал бүгдийг харах ёстой. 1 000 элементтэй гурван жагсаалт дээр тоолуулсан нь:

Ролт	Харьцуулалт	Жинхэнэ солилт
эрэмбэлэгдсэн	499 500	0
санамсаргүй	499 500	995
урвуу	499 500	500

Хамгийн сайн, дундаж, хамгийн муу — гурвуулаа **O(n²)**. Бодитоор хэмжиж шалгая:



Зураг 2. selection_sort-ийн хугацаа, санамсаргүй жагсаалт. Энэ PDF-ийг үүсгэх үед бодитоор хэмжсэн (3 давталтын хамгийн бага утга). Таны компьютер дээр тоо нь өөр гарах ч хэлбэр нь ижил — шулуун биш, дээшээ **муруйсан** шугам.

1 000 элементэд **8 мс**, 2 000 элементэд **32 мс** — **3.9 дахин**. O(n²): n 2 дахин → ажил 2^2 = 4 дахин. Энэ хурдаар 1 сая элемент эрэмбэлэхэд ойролцоогоор **2.2 цаг** шаардлагатай (тооцоолсон). Харьцуулахад Python-ий `sorted()` сая элементийг **123 мс**-д эрэмбэлэв (хэмжсэн) — яагаад гэдгийг 3/3 хичээлд үзнэ.

Bubble sort-ийг сонссон уу?

Хөрш хоёр элементийг харьцуулж, буруу дараалалтай бол солино; том нь «бөмбөлөг» шиг төгсгөл рүү хөвнө. Энэ ч бас O(n²):

```
def bubble_sort(items):
    n = len(items)
    for i in range(n - 1):
        for j in range(n - 1 - i):
            if items[j] > items[j + 1]:
                items[j], items[j + 1] = items[j + 1], items[j]
```

Харьцуулалтын тоо ижил (499 500), гэхдээ 1 000 элементтэй санамсаргүй жагсаалтад bubble sort **248 714** удаа, selection sort ердөө **995** удаа солив. Солих үйлдэл цөөн нь selection sort-ийн давуу тал.

07 Тестлэх

Эрэмбэлэлтийг тестлэх хамгийн хялбар арга: Python-ий `sorted()`-ийг «зөв хариу» болгож, мянга мянган санамсаргүй жагсаалт дээр харьцуулах.

```
import random

def test_random(trials=10_000):
    for _ in range(trials):
        items = [random.randint(0, 9)           # 0–9: давхардал их гарна
                 for _ in range(random.randint(0, 12))]
        expected = sorted(items)                # «зөв хариу»
        selection_sort(items)                   # байрандаа эрэмбэлнэ
        assert items == expected, expected
    print(trials, "санамсаргүй тест: OK")
```

Захын тохиолдлуудыг (edge cases) тусад нь шалгана — алдаа ихэвчлэн хил дээр нуугддаг:

```
def test_edges():
    for items in ([], [5], [1, 2, 3], [3, 2, 1], [7, 7, 7], [2, 1, 2, 1]):
        #         хоосон, ганц, эрэмбэлэгдсэн, урвуу, бүгд ижил, давхардалтай
        expected = sorted(items)
        selection_sort(items)
        assert items == expected, expected
    print("Захын тохиолдлууд: OK")
```

Энэ PDF-ийг үүсгэх үед зөв хувилбарыг **10 000** санамсаргүй тестээр шалгахад бүгд амжилттай давлаа. Харин 5-р бүлгийн алдаатай хувилбаруудыг ижил тестэд оруулахад:

Хувилбар	1 000 тестээс бүтэлгүйтсэн	Хамгийн богино жишээ	Алдааны төрөл
Алдаа 1: `min_i = 0`	**737**	`[0, 2, 3] → [2, 0, 3]`	буруу хариу
Алдаа 2: `items[j] < items[i]`	**512**	`[5, 1, 2] → [2, 1, 5]`	буруу хариу
Зөв `selection_sort`	**0**	—	—

Дүгнэлт
1 000 тестээс зөв хариу өгсөн нь: алдаа 1 — **263**, алдаа 2 — **488**. Алдаатай код ч хоосон, ганц элементтэй эсвэл аз таарсан жагсаалт дээр «зөв» ажилладаг. Тиймээс ганц хоёр жишээгээр биш, санамсаргүй олон тестээр шалга. «Хамгийн богино жишээ» баганын жагсаалтыг гараар мөрдвөл алдаа хурдан ойлгогдоно.

08 Дасгал

Эхлээд өөрөө бодож, кодоо 7-р бүлгийн аргаар тестлээрэй. Хариуг дараагийн хуудаснаас шалга.

Дасгал 1 · Гараар мөрдөх

`selection_sort([5, 1, 4, 2, 3])`: эргэлт бүрийн дараах жагсаалтыг бич. Нийт хэдэн харьцуулалт, хэдэн жинхэнэ солилт хийх вэ?

Дасгал 2 · Буурахаар

`selection_sort_desc(items)` бич: жагсаалтыг **буурахаар** эрэмбэлнэ. Кодонд хамгийн бага өөрчлөлт хэд вэ?

```
nums = [3, 8, 1, 9, 4]
selection_sort_desc(nums)
print(nums)                # [9, 8, 4, 3, 1]
```

Дасгал 3 · Тооцоо

6 элементтэй санамсаргүй жагсаалтад selection sort хэдэн харьцуулалт хийх вэ? Жагсаалт аль хэдийн эрэмбэлэгдсэн байвал? 100 элементтэй бол?

Дасгал 4 · Алдааг ол

Доорх код `[3, 2, 1]`-ийг юу болгох вэ? Яагаад? Хэрхэн засах вэ?

```
def selection_sort(items):
    n = len(items)
    for i in range(n - 1):
        min_i = i
        for j in range(i + 1, n - 1):
            if items[j] < items[min_i]:
                min_i = j
        items[i], items[min_i] = items[min_i], items[i]
```

Дасгал 5 · Хамгийн бага k

`smallest_k(items, k)`: хамгийн бага k элементийг өсөхөөр буцаа. Selection sort-ийн зөвхөн эхний k эргэлтийг хий; эх жагсаалтыг өөрчилж болохгүй. Цагийн төвөгшил нь хэд вэ?

```
smallest_k([29, 10, 14, 37, 13], 2)    # [10, 13]
```

Дасгалын хариу

Бүх хариуг энэ PDF-ийг үүсгэх үед жишээн дээр нь ажиллуулж шалгасан.

1. Гараар мөрдөх — 10 харьцуулалт, 4 солилт

i	min (индекс)	Эргэлтийн дараа
0	1 (1)	`[1, 5, 4, 2, 3]`
1	2 (3)	`[1, 2, 4, 5, 3]`
2	3 (4)	`[1, 2, 3, 5, 4]`
3	4 (4)	`[1, 2, 3, 4, 5]`

Энэ жишээнд эргэлт бүрт жинхэнэ солилт гарсан: 4 = n − 1 — selection sort-ийн солилтын дээд хязгаар. Харьцуулалт ч мөн үргэлжийн адил n(n − 1) / 2 = 10.

2. Буурахаар — ганц тэмдэгт

```
def selection_sort_desc(items):
    n = len(items)
    for i in range(n - 1):
        max_i = i                          # одоо хамгийн ИХийг хайна
        for j in range(i + 1, n):
            if items[j] > items[max_i]:    # <  →  >
                max_i = j
        items[i], items[max_i] = items[max_i], items[i]
```

3. Тооцоо

6 · 5 / 2 = **15**. Эрэмбэлэгдсэн байсан ч мөн **15** — selection sort оролтоос хамаарахгүй. 100 элементэд 100 · 99 / 2 = **4 950**.

4. Алдааг ол — `[2, 3, 1]`

`range(i + 1, n - 1)` нь сүүлийн индекс n − 1-ийг хэзээ ч шалгахгүй. [3, 2, 1]-д хамгийн бага 1 нь яг сүүлд байгаа тул хэзээ ч сонгогдохгүй. Засвар: `range(i + 1, n)`. Энэ бол Хайлт 1/2-оос танил «нэгээр зөрөх» (off-by-one) алдаа.

5. Хамгийн бага k — O(k · n)

```
def smallest_k(items, k):
    a = items[:]                           # хуулбар: эх жагсаалт өөрчлөгдөхгүй
    n = len(a)
    for i in range(k):                     # n - 1 биш, ердөө k эргэлт
        min_i = i
        for j in range(i + 1, n):
            if a[j] < a[min_i]:
                min_i = j
        a[i], a[min_i] = a[min_i], a[i]
    return a[:k]
```

k жижиг бол бүтэн эрэмбэлэхээс (O(n²)) хамаагүй хурдан. Selection sort-ийн онцлог: k эргэлтийн дараа эхний k элемент **эцсийн** байрандаа орсон байдаг.

09 Хураангуй

Загвар — цээжлэх биш, ойлгож бич

Selection sort

```
def selection_sort(items):
    n = len(items)
    for i in range(n - 1):
        min_i = i
        for j in range(i + 1, n):
            if items[j] < items[min_i]:
                min_i = j
        items[i], items[min_i] = \
            items[min_i], items[i]
```

Гол тоонууд

Харьцуулалт	**n(n − 1) / 2 — үргэлж**
Солилт	**≤ n − 1**
Цаг (сайн / муу)	**O(n²) / O(n²)**
Санах ой	**O(1) — байрандаа**
Тогтвортой юу?	**Үгүй (2/3-т үзнэ)**

Selection sort бичихдээ шалгах 4 зүйл

1. `min_i = i` — хайлт **эрэмбэлэгдээгүй** хэсгээс эхэлж байна уу?
2. `items[min_i]`-тэй (одоогийн хамгийн багатай) харьцуулж байна уу?
3. `range(i + 1, n)` — сүүлийн элементийг хамарч байна уу?
4. Солилт дотоод давталтын **гадна**, эргэлтэд нэг удаа байна уу?

Санаж үлдэх зүйлс

- Зүүн хэсэг — эрэмбэлэгдсэн, эцсийн байрандаа; баруунаас хамгийн багыг **сонгож** залгана.
- Цаг: үргэлж **O(n²)**; n 2 дахин → хугацаа 4 дахин. Санах ой: O(1).
- `sort()` ба бидний функц байрандаа эрэмбэлж `None` буцаана; шинэ жагсаалт — `sorted()`.
- Эрэмбэлэлтийг `sorted()`-тэй харьцуулсан санамсаргүй тестээр шалга.

> **Дараагийн алхам: Insertion sort**
> Хөзөр тоглохдоо гартаа ирсэн хөзрийг эрэмбэлсэн хөзрүүдийнхээ дунд зөв байранд нь оруулдаг. Энэ санаа аль хэдийн бараг эрэмбэлэгдсэн жагсаалтад **маш хурдан** — «Эрэмбэлэлт 2/3: Insertion sort» хичээлийн сэдэв.